

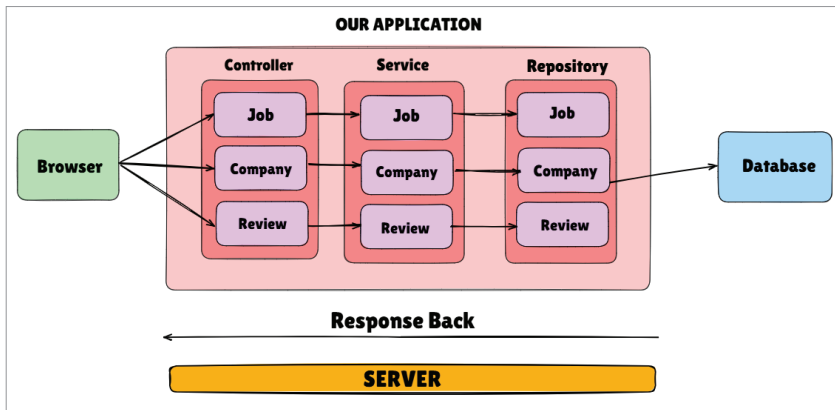


Figure 4: Job portal - monolithic application

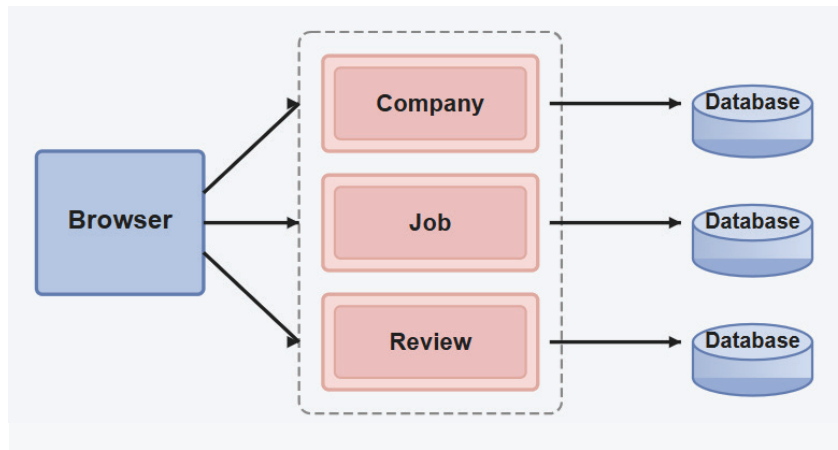


Figure 5: Job portal – microservices application

consistency models such as eventual consistency and CAP theorem principles were considered to manage data integrity across distributed databases. These models provided a structured approach to making architectural decisions, ensuring that the transition maintained high system reliability, scalability, and responsiveness.

Technology stack

Spring Boot was chosen for implementing the microservices architecture due to its robust support for building scalable, production-ready applications with minimal configuration. Prior to this transition, the system followed a monolithic architecture, where all functionalities were tightly coupled within a single application as shown in Figure 4. It

provides a lightweight framework that simplifies the development of individual services, allowing each microservice—such as Job, Company, and Review—to be developed, tested, and deployed independently as presented in Figure 5. Spring Boot’s seamless integration with Spring Cloud offers powerful tools for service discovery (via Eureka), centralised configuration, load balancing (using Traefik), and fault tolerance (with Hystrix or Resilience4j). Moreover, its built-in support for RESTful APIs, embedded servers like Tomcat, and easy integration with databases makes it an ideal choice for microservices. These features significantly accelerated development while ensuring consistency, flexibility, and maintainability across the entire system.

Technology stack for the microservices-based open source job portal

Component	Technologies
Frontend	React, Next.js
Backend	Java, Spring Boot
Database	PostgreSQL
Containerisation	Docker
Monitoring/Tracing	Zipkin
Message queue	RabbitMQ
Service discovery	Eureka
Load balancer	Traefik

Folder and file structure

Figure 6 describes the project structure of the microservices application, which is modularized as follows:

- **dto/**: Contains Data Transfer Objects for efficient data communication between services.
- **external/**: Handles inter-service communication using REST clients.
- **impl/**: Contains business logic implementation.
- **Job.java**: Defines the Job entity mapped to the database.
- **JobController.java**: Manages HTTP endpoints for Job-related operations.
- **JobRepository.java**: Provides CRUD operations for the Job entity.
- **JobService.java**: Contains core job-related business logic.

DTO (Data Transfer Object)

DTO stands for Data Transfer Object. It is an object that is used to transfer data between software application subsystems or layers, typically over a network. DTOs are designed to be simple containers for data, and they do not contain any business logic. They are primarily used to reduce the number of method calls by encapsulating multiple data elements in a single object. For example, when transferring data from a database to a client application, a DTO can aggregate various pieces of information (such as a user’s name, email, and address) into a single object. This helps optimise communication, especially in distributed